Algorithm Complexity: Midterm Cheat Sheet + Explanation

DSA Labs 1–4 | Java | Arrays, searching, sorting, sliding windows, stacks, queues, linked lists, recursion

Read this before the tables. A complexity bound describes how the *number of basic operations* or *memory used* grows as input becomes larger. It is not the exact runtime in seconds. Unless a row says otherwise, time is **worst-case**; "space" is shown as **stored data** and/or **extra (auxiliary) memory**. All bounds assume ordinary constant-time primitive operations.

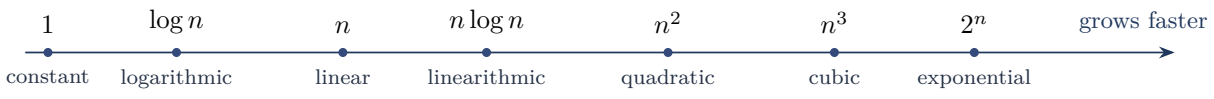
1. Basic concepts: what are time and space complexity?

Term	Student-friendly definition
Input size n	Number of elements, requests, characters, nodes, etc. Some problems have two independent sizes n, m, a window width k, or range R.
Time complexity $T(n)$	How the amount of work grows with input size. Count comparisons, assignments, pointer updates, and other basic operations, not the stopwatch time.
Total space	Memory for input/storage, output, and temporary work (including active recursive stack frames).
Auxiliary space	*Extra working memory* beyond input storage; state separately whether required output is counted.
Best / average / worst	Least / expected-under-stated-distribution / most work among inputs of size n. Compare algorithms using the same case.
Amortized time	Per-operation cost averaged over a *sequence* of operations; e.g., a dynamic array occasionally resizes, but append is amortized $O(1)$.
Expected time	Average over randomness or hashing assumptions; not necessarily a guaranteed worst-case bound.
In-place	Uses $O(1)$ or little auxiliary storage, depending on the convention; always state the actual bound.

Big O, Ω, and Θ. $O(g(n))$ means an **asymptotic upper bound**, $\Omega(g(n))$ means a **lower bound**, and $\Theta(g(n))$ means a **tight bound** (both). E.g., optimized bubble sort has best-case $\Theta(n)$ and worst-case $\Theta(n^2)$. The case (best/worst) and notation (O/Θ) describe different things.

2. How to compare two time complexities

Growth class	How it grows	Typical course example
$O(1)$	No growth with n	Read array element; stack top
$O(\log n)$	Add one step when n roughly doubles	Binary search in indexed sorted array
$O(\sqrt{n})$	Much slower growth than linear	Some divisor checks (context-dependent)
$O(n)$	Roughly doubles with double input	Linear scan, traversal, iterative sum
$O(n \log n)$	Somewhat more than doubles	Efficient comparison sorting (extension)
$O(n^2)$	Roughly quadruples with double input	Bubble / insertion worst case / selection
$O(n^3)$	Roughly eight times as input doubles	Three full nested loops
$O(2^n)$	Exponentially many states	Naive recursive Fibonacci upper bound
$O(n!)$	Extremely fast growth	Enumerating all permutations



Ordering for sufficiently large n: $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$. **Caution:** This

ranking ignores constant factors, memory use, and smaller practical input sizes. A smaller Big-O class does not guarantee faster execution for every n.

Compare them numerically (rough operation counts)

n	$\log_2 n$	n	$n \log_2 n$	n^2
10	≈ 3.3	10	≈ 33	100
100	≈ 6.6	100	≈ 664	$10,000$
$1,000$	≈ 10	$1,000$	$\approx 9,966$	$1,000,000$
$10,000$	≈ 13.3	$10,000$	$\approx 132,877$	$100,000,000$

Comparing method (exam checklist): (1) Define n and any m, k, R; (2) count fundamental operations; (3) express best/average/worst if relevant; (4) remove constant factors and lower-order terms; (5) compare growth at the *same* input size; (6) compare auxiliary memory and required preconditions; (7) mention real-world tradeoffs when both bounds match.

3. How to determine complexity from Java code

Constant and linear

```
int x = a[0];                    // O(1)
for (int i=0; i<n; i++) sum++; // O(n)
```

Two consecutive loops

```
for (int i=0; i<n; i++) work();
for (int j=0; j<m; j++) work();
// O(n + m), NOT O(n*m)
```

Full nested loops

```
for (int i=0; i<n; i++)
    for (int j=0; j<n; j++) work();
// O(n*n) = O(n^2)
```

Triangular nested loops

```
for (int i=0; i<n; i++)
    for (int j=0; j<i; j++) work();
// 0+1+...+(n-1) = n(n-1)/2
// Theta(n^2)
```

Repeated halving / doubling

```
while (n > 1) n /= 2; // O(log n)
for (int i=1; i<n; i*=2) work();
// O(log n)
```

Nested loops are not automatically quadratic. If two pointers only move forward across an array, their *total* movements may be $O(n)$ even with a nested `while`. The same amortized reasoning explains monotonic stack/deque algorithms: each element is inserted and removed at most once.

Dominant-term simplification. $7n^2 + 3n + 18 = \Theta(n^2)$; $n(n-1)/2 = \Theta(n^2)$; $n + m$ must retain both variables unless their relation is known. When complexity contains R or k, define it before using a formula.

4. Sorting algorithms (the three studied sorts)

Algorithm	Best	Average	Worst	Aux. space	Stable?	Key reason
Bubble, early-exit	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes	Adjacent swaps; an entire pass without swaps ends early.
Bubble, without early-exit	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes	Always makes all comparison passes.
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes	Shift larger values to insert the next key; fast on nearly sorted inputs.
Selection (standard)	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	No	Scan entire remaining suffix for the minimum.

Stable = equal-key items preserve original relative order. Standard bubble/insertion are stable when comparisons are strict; ordinary selection with swapping is not generally stable.

Trace the *first pass* on $[5, 2, 4, 1]$

Sort	What happens in this pass	After pass 1
Bubble	Swap adjacent out-of-order neighbors: $[2, 5, 4, 1] \to [2, 4, 5, 1] \to [2, 4, 1, 5]$.	$[2, 4, 1, 5]$: largest value is fixed at end.
Insertion	Insert the second element 2 into sorted prefix $[5]$: move 5 right, place 2.	$[2, 5, 4, 1]$: sorted prefix length 2.
Selection	Find minimum 1 at index 3; swap with index 0.	$[1, 2, 4, 5]$: minimum fixed at front.

Where $O(n^2)$ comes from: a descending input can require roughly $n(n-1)/2$ comparisons/shifts. Selection does about this many comparisons even if the input is already sorted. All these bounds exclude the preexisting storage for the n array items.

5. Arrays, array lists, searching, and sliding windows

Array and Java `ArrayList` operations

Operation	Array `int[]`	`ArrayList`	Assumption / detail
Read/set item at known index	$O(1)$	$O(1)$	Valid index.
Traverse/print all elements	$O(n)$	$O(n)$	Printing cost counted.
Find value in unsorted data	$O(n)$	$O(n)$	Linear search; best case $O(1)$.
Append at end	Not resizable; copy $O(n)$	Amortized $O(1)$	`ArrayList` occasional growth copy $O(n)$.
Remove last element	Logical size change $O(1)$	$O(1)$	Fixed Java arrays cannot actually shrink.
Insert/delete near front or middle	$O(n)$	$O(n)$	Move/shift later elements.
Copy an array/list of n items	$O(n)$	$O(n)$	Destination storage $O(n)$.

Storage: fixed array of length n uses $\Theta(n)$ cells; dynamic array uses $O(C)$ allocated cells for capacity $C \geq n$, typically $O(n)$ under geometric growth. The extra working space for ordinary indexing/searching is $O(1)$.

Searching

Method	Best time	Worst time	Aux. space	Condition / note
Linear scan	$\Theta(1)$	$\Theta(n)$	$O(1)$	Unsorted or sorted data; may stop early.
Binary search, iterative	$\Theta(1)$	$\Theta(\log n)$	$O(1)$	**Sorted random-access** array; maintain inclusive interval.
Binary search, recursive	$\Theta(1)$	$\Theta(\log n)$	$O(\log n)$	Same comparisons; recursive stack depth.
Leftmost/rightmost duplicate	$O(\log n)$	$O(\log n)$	$O(1)$ iterative	Keep shrinking after candidate is found.
Binary search on answer	$O(C(n))$ check	$O(C \log R)$	Check-dependent	Monotone predicate; C is check cost and R the search-range size.

Binary search trace: sorted $[2, 4, 6, 8, 10, 12, 14]$, target 10. $(low, high) = (0, 6)$, $mid = 3$, $a[mid] = 8 < 10$, keep $(4, 6)$; $mid = 5$, $12 > 10$, keep $(4, 4)$; $mid = 4$, $a[4] = 10$, found. Three checks. For 11, the final interval becomes empty: return -1.

Critical caveat: A linked list does not offer $O(1)$ indexed midpoints. Its ordinary search costs $O(n)$, so "use binary search on a linked list" does not give array-like $O(\log n)$ time.

Sliding windows, two pointers, prefix sums

Method	Total time	Aux. space	When valid / why
Recalculate each window sum, width k	$O((n-k+1)k)$	$O(1)$	Read all k values again per window.
Fixed-width rolling sum/average	$O(n)$	$O(1)$	Remove outgoing value, add incoming one.
Variable window, two forward pointers	$O(n)$	$O(1)$ or state-dependent	Both pointers only advance; condition must allow valid shrink/expand reasoning.
Prefix-sum construction	$O(n)$	$O(n)$	Array of cumulative sums.
One sum query after prefix preprocessing	$O(1)$	$O(1)$ extra per query	The $O(n)$ prefix table is already built.
Sliding-window maximum, naive	$O((n-k+1)k)$	$O(1)$	Scan each full window.
Sliding-window maximum, monotonic deque	$O(n)$	$O(k)$	Store candidate *indices*; each enters/exits once.
Window frequency via hash map	Expected $O(n)$	$O(k)$ (or alphabet)	Expected $O(1)$ hash updates; depends on what is stored.

For window maxima with an output array, add $O(n-k+1)$ output space. Variable windows for sum thresholds may be invalid with negative values: reason about the condition first.

6. Stacks: all course variants and common applications

LIFO: the most recently pushed item is removed first. For the operations below, `size` and `isEmpty` are $O(1)$ when a size counter is maintained.

Variant	Operation costs	Stored space and caveat
Fixed-capacity array stack	Push, pop, peek: $O(1)$	$O(K)$ capacity; a full stack cannot grow.
Growing array stack	Push: amortized $O(1)$, worst $O(n)$ when resizing; pop/peek: $O(1)$	$O(n)$ with geometric capacity growth.
Singly linked stack	Push, pop, peek: $O(1)$	$O(n)$ nodes; operate at the head.
Java `ArrayDeque` as a stack	Push/pop: amortized $O(1)$; peek: $O(1)$	$O(n)$; may reallocate; no null entries.
Min stack (two stacks/paired minima)	Push, pop, peek, `getMin`: $O(1)$	$O(n)$ values and tracked minima; duplicate minima matter.
Two-stack undo/redo	Edit, undo, redo: $O(1)$ per action	$O(n)$ history; new edits clear redo history.

Applications using a stack

Application	Time; auxiliary space	Reason
Bracket matching over n characters	$O(n)$; $O(n)$	Read once; nesting depth can reach n.
Reverse n items with an extra stack	$O(n)$; $O(n)$	Each item goes in and out once.
Next greater element (monotonic stack)	$O(n)$; $O(n)$	Each index pushed once and popped at most once.
Find a value by scanning stack contents	$O(n)$; $O(1)$ extra	No indexed constant-time search guarantee.

Monotonic stack intuition: Process $[2, 5, 3, 6]$. The new value 5 resolves the waiting 2; 3 waits; 6 resolves both 3 and 5. One iteration can pop many entries, but no index is popped twice. **Total** $O(n)$, not $O(n^2)$.

7. Queues, deques, heaps, and scheduling

FIFO: ordinary queues serve the oldest enqueued item; a *priority* queue serves by priority instead. Maintained `size` / `isEmpty` queries take $O(1)$.

Variant	Enqueue / dequeue / peek	Stored memory and notes
Naive array, shift elements left on dequeue	$O(1)$ / $O(n)$ / $O(1)$	$O(K)$ capacity; the cost comes from shifting.
Fixed circular-array queue	$O(1)$ / $O(1)$ / $O(1)$	$O(K)$; modular indices reuse positions.
Linked queue (head and tail)	$O(1)$ / $O(1)$ / $O(1)$	$O(n)$ nodes; clear tail when the last node leaves.
Java `ArrayDeque` queue	Amortized $O(1)$ / amortized $O(1)$ / $O(1)$	$O(n)$; resizing can be costly once.
Deque, fixed ring or doubly linked	Add/remove/peek at *either end*: $O(1)$	$O(K)$ ring or $O(n)$ nodes. Growable deques: amortized $O(1)$ ends.
Binary heap priority queue	Insert $O(\log n)$ / extract $O(\log n)$ / peek $O(1)$	$O(n)$ heap; equal-priority FIFO requires an explicit tie-breaker.

Specialized queue operations

Operation / algorithm	Time; extra memory	Details
Heap `size`, `isEmpty`, `peek`	$O(1)$; $O(1)$	Top item is directly accessible.
Heap `contains(x)` or remove arbitrary object	$O(n)$ worst; $O(1)$ extra	May need a linear search for that object.
Bottom-up heapify of n items	$O(n)$; $O(1)$ extra possible	Different from n separate heap insertions.
Monotonic deque (sliding maximum), width k	$O(n)$ total; $O(k)$	Each candidate index enters/leaves once; output adds $O(n-k+1)$ if stored.
Traverse/print n queue items	$O(n)$; $O(1)$ iterator	Printing all values already costs $O(n)$.

The similar-sounding names mean different things. Circular queue = FIFO using wrapped array indices. Circular linked list = pointers form a cycle. Deque = both ends accessible. Monotonic deque = maintain only non-dominated window candidates. Priority queue = remove according to priority, typically with a heap.

8. Linked lists: all relevant variants and operation caveats

Assume **head and tail references** are maintained. n is the node count. A "known node" means a direct reference; singly-linked deletion typically also needs the predecessor.

Operation	Singly H+T	Doubly H+T	Circular singly + tail
Get by index / search for value	$O(n)$	$O(n)$	$O(n)$
Add first / add last	$O(1)$ / $O(1)$	$O(1)$ / $O(1)$	$O(1)$ / $O(1)$
Remove first / remove last	$O(1)$ / $O(n)$	$O(1)$ / $O(1)$	$O(1)$ / $O(n)$
Remove known interior node	$O(1)$ with predecessor	$O(1)$ with node	$O(1)$ with predecessor
Remove first matching value	$O(n)$	$O(n)$	$O(n)$
Traverse all nodes / print	$O(n)$	$O(n)$	$O(n)$
Reverse links iteratively	$O(n)$, $O(1)$ aux.	$O(n)$, $O(1)$ aux.	$O(n)$, $O(1)$ aux.
Stored node space	$O(n)$	$O(n)$	$O(n)$

Circular doubly linked list: With direct head/tail/node references, insert/remove at either end and remove a known node all take $O(1)$. Search and full traversal remain $O(n)$; storage $O(n)$. If a *singly* list stores only head but not tail, tail append becomes $O(n)$.

Linked-list algorithms and further distinctions

Algorithm	Time; auxiliary space	Why / qualification
Find middle with slow/fast pointers	$O(n)$; $O(1)$	Pointers move along references.
Floyd's cycle detection	$O(n)$; $O(1)$	Compare *node identity*, not equal values.
Cycle detection with hash set	Expected $O(n)$; $O(n)$	Store visited node identities.
Reverse a list recursively	$O(n)$; $O(n)$	Recursive call frames consume memory.
Step to known `next` (or `prev`)	$O(1)$; $O(1)$	One direct pointer hop.
Linked queue: enqueue / dequeue	$O(1)$ each; $O(1)$ per call	Maintain head and tail; stored nodes $O(n)$.

Exam trap: "Linked-list deletion is $O(1)$." Only when the required node/neighbor references are *already known*. Deleting by *value* normally takes $O(n)$ searching. A singly linked list cannot normally delete the tail in $O(1)$ without a predecessor reference.

9. Space: stored data versus extra memory

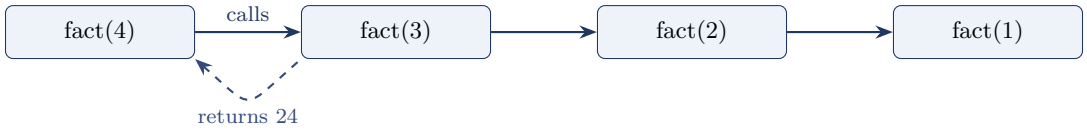
Example	Storage for data	Auxiliary space for the operation/algorithm
Linear array scan	$\Theta(n)$ input	$O(1)$ counters.
Bubble / insertion / selection sort	$\Theta(n)$ array	$O(1)$ work storage.
Linked list (all four variants)	$\Theta(n)$ nodes	$O(1)$ for a normal local link change.
Fixed circular queue, capacity K	$\Theta(K)$ buffer	$O(1)$ for enqueue/dequeue.
Stack of n items	$\Theta(n)$	$O(1)$ for pop/peek (without resize).
Monotonic deque window maximum	$\Theta(n)$ input	$O(k)$ deque; output $O(n-k+1)$ if retained.
Recursive scan over n items	$\Theta(n)$ input	$O(n)$ active stack frames.

10. Recursion: time and call-stack space

Recursion means a method calls itself on a smaller subproblem. It requires a valid **base case**, **progress toward it**, and a way to combine results. Each active call normally occupies a Java stack frame; Java does not guarantee tail-call elimination.

Recursive algorithm	Time	Stack space	Recurrence / explanation
Factorial / sum 1 through n	$\Theta(n)$	$\Theta(n)$	$T(n) = T(n-1) + O(1)$.
Recursive sum or linear search	$O(n)$	$O(n)$	One smaller call per level.
Recursive binary search	$O(\log n)$	$O(\log n)$	$T(n) = T(n/2) + O(1)$.
Naive Fibonacci, two branches	$O(2^n)$ (tighter $\Theta(\varphi^n)$)	$O(n)$	$T(n) = T(n-1) + T(n-2) + O(1)$.
Memoized Fibonacci	$O(n)$	$O(n)$	Each state computed once; cache plus frames.
Divide, then combine in linear time	$\Theta(n \log n)$	Implementation-dependent	$T(n) = 2T(n/2) + O(n)$; a merge needs $O(n)$ scratch.

Call-stack trace for factorial(4):



Unwind: $1 \to 2 \to 6 \to 24$; 4 frames can be active, so space $O(n)$.

Iterative sum (linear time, constant extra space)

```
static int sum(int[] a) {
    int s = 0;
    for (int x : a) s += x;
    return s;
}
// T(n) = O(n), auxiliary S(n) = O(1)
```

Recursive sum (same time, extra call stack)

```
static int sum(int[] a, int i) {
    if (i == a.length) return 0;
    return a[i] + sum(a, i + 1);
}
// T(n) = O(n), auxiliary S(n) = O(n)
```

How to analyze recursion: (1) Count calls made at each step; (2) state the work done outside recursive calls; (3) write a recurrence; (4) calculate how many levels/branches exist; (5) calculate *maximum simultaneously active* frames for space. Total calls and maximum stack depth are not the same quantity.

11. Combining costs: real midterm-style questions

Task	Combined complexity
Sort n numbers with insertion sort, then binary-search once	Worst $O(n^2) + O(\log n) = O(n^2)$; space $O(1)$ aux.
Sort n numbers once with an $O(n \log n)$ sort, then answer q binary searches	$O(n \log n + q \log n)$ total; record sorting's auxiliary space separately.
Loop through n items and for each perform an $O(m)$ scan	$O(nm)$, not $O(n + m)$.
Read all n elements, then all m elements	$O(n + m)$, not $O(nm)$.
Process n incoming values with a monotonic deque	$O(n)$ total, because each index enters and leaves at most once.
Perform q heap insertions/removals while heap size is at most n	$O(q \log n)$ total for these operations; storage $O(n)$.